



Faculdades de Campinas

ENGENHARIA DA COMPUTAÇÃO

Tratamento e Armazenamento de Informação

Atividade 3.2 — Questões Dissertativas: Projeto Prático NoSQL com Apache CouchDB

GRUPO:

Felipe Furlan RA 202210339

Henrique Biruel RA 202210347

Henrique Palorca RA 202210348

Campinas
Setembro de 2026

1 INTRODUÇÃO

O presente trabalho apresenta as respostas às dez questões dissertativas propostas na aula sobre o projeto prático de banco NoSQL da disciplina Tratamento e Armazenamento de Informação. O material de referência conduz a construção de um MVP de comércio eletrônico com Python, Flask e Apache CouchDB, percorrendo requisitos e padrões de acesso, modelagem documental, criação do database e dos índices Mango, integração da aplicação por REST/JSON, tratamento de concorrência e de conflitos, plano de testes, segurança, backup, replicação e implantação em nuvem (MARCUSO, 2026). A discussão desenvolvida a seguir toma como fio condutor a ideia central do projeto: a escolha de um banco de documentos precisa ser justificada pelo padrão de acesso e assumida com seus compromissos, uma vez que garantias oferecidas pelo servidor no modelo relacional passam a depender de decisões explícitas de modelagem e de código.

2 QUESTÃO 1 — JUSTIFICATIVA DO COUCHDB PARA O PROJETO

Justifique CouchDB para o projeto.

A adoção de um banco NoSQL deve ser justificada pelo padrão de acesso da aplicação, e não pela tecnologia em si (MARCUSO, 2026). O projeto é um MVP de comércio eletrônico cujo catálogo possui estrutura variável — atributos que diferem entre categorias e evoluem com frequência — e cujas leituras principais recuperam agregados inteiros: a página de um produto, o pedido com todos os seus itens e o histórico de compras de um cliente. Esse padrão corresponde exatamente à unidade do modelo documental, um objeto JSON autodescritivo, lido e gravado em uma única operação, sem necessidade de junções entre estruturas separadas.

Além do modelo de dados, quatro características do CouchDB atendem a requisitos declarados do projeto. A API REST nativa dispensa driver específico e reduz a integração com o Flask a chamadas HTTP com JSON, aproximando o código daquilo que se inspeciona visualmente no Fauxton. O controle de concorrência otimista por `_rev` expõe no próprio protocolo o conflito entre atualizações simultâneas, tema que no modelo relacional permanece oculto atrás da transação. A replicação embutida oferece caminho direto para cópia, recuperação e cenários de operação desconectada. Por fim, tratando-se de projeto Apache sob licença Apache 2.0,

executável localmente em contêiner, o custo do laboratório permanece nulo e a portabilidade para a nuvem é preservada.

Uma justificativa honesta precisa incluir os limites. O CouchDB não oferece atomicidade entre documentos distintos, de modo que o checkout — que grava o pedido e altera vários produtos — exige estratégia explícita de consistência; e as consultas dependem de índices criados deliberadamente para cada padrão de acesso, sob pena de varredura completa do database. Para uma loja de porte pequeno, com catálogo enxuto e fluxo de compra simples, esse custo é aceitável e didaticamente produtivo. Para um sistema com forte necessidade transacional, como conciliação financeira ou controle contábil, a escolha adequada continuaria sendo o banco relacional. A decisão é, portanto, de adequação ao problema, e não de superioridade de uma tecnologia sobre a outra.

3 QUESTÃO 2 — DOCUMENTO E TABELA: COMPARAÇÃO ENTRE OS MODELOS

Compare documento e tabela.

A diferença essencial está em onde reside a estrutura dos dados. No modelo relacional, o esquema é declarado no banco: tabelas, colunas tipadas, chaves e restrições são verificadas no momento da escrita, e a normalização distribui uma entidade por várias tabelas a fim de eliminar redundância e anomalias de atualização (ELMASRI; NAVATHE, 2011). No modelo documental, a estrutura reside dentro do próprio documento, que é autodescritivo: cada documento carrega seus campos, admite listas e objetos aninhados e não precisa ser idêntico aos demais armazenados no mesmo database. O banco exige pouco na escrita e transfere para a aplicação a responsabilidade de manter a coerência entre documentos relacionados.

Dessa diferença decorrem consequências práticas em quase todas as etapas do projeto. A leitura passa a ocorrer por agregado, em uma única requisição, em vez de ser composta por junções. A evolução do esquema deixa de exigir migração, pois novos campos convivem com documentos antigos — o que, em contrapartida, obriga a versionar o formato para que o código saiba interpretar as duas gerações. O relacionamento deixa de ser garantido por chave estrangeira e passa a ser expresso por incorporação ou por referência resolvida no código. A concorrência, por fim, deixa

de ser controlada por uma transação abrangente e passa a ser controlada documento a documento, pela revisão. O Quadro 1 sintetiza a comparação.

Quadro 1 — Comparação entre o modelo relacional em tabelas e o modelo documental do CouchDB

Critério	Tabela (PostgreSQL)	Documento (CouchDB)
Local do esquema	Declarado no banco e validado na escrita	Dentro do documento, validado pela aplicação
Unidade de leitura	Linhas combinadas por junção	Agregado completo em uma requisição
Relacionamento	Chave estrangeira e JOIN	Incorporação ou referência resolvida no código
Evolução do formato	Alteração de tabela e migração de dados	Novos campos convivem; exige versionamento
Atomicidade	Transação sobre várias linhas e tabelas	Limitada a um documento, com controle por <code>_rev</code>
Consulta	SQL declarativo com otimizador	Seletor Mango sobre índices criados por consulta

Fonte: elaborado pelos autores com base em Marcusso (2026) e em Elmasri e Navathe (2011).

O pedido do projeto ilustra bem o contraste. Na versão relacional, ele se divide em duas tabelas, pedido e item_pedido, ligadas por chave estrangeira, e a consulta do histórico exige junção; o banco impede, por construção, que exista item sem pedido correspondente. Na versão documental, o pedido é um único documento com o vetor de itens, gravado e lido integralmente, no qual nome e preço são copiados como registro histórico da compra. A segunda forma responde ao histórico em uma requisição e preserva o valor praticado na data; em compensação, nada no banco impede gravar um item cujo produto não exista, de modo que a integridade referencial passa a ser responsabilidade do código de checkout.

4 QUESTÃO 3 — EMBED E REFERENCE NA MODELAGEM DOCUMENTAL

Análise embed x reference.

A decisão entre incorporar e referenciar é a escolha central da modelagem documental. O critério consolidado é o ciclo de vida do dado: incorpora-se aquilo que é lido e alterado junto com o documento principal e que não possui existência independente; referencia-se aquilo que tem vida própria, é compartilhado por muitos documentos ou cresce sem limite previsível (SADALAGE; FOWLER, 2012). Como o documento é também a fronteira da atomicidade no CouchDB, essa decisão

determina simultaneamente o custo das consultas e o alcance das garantias de escrita. O Quadro 2 registra as decisões adotadas no projeto.

Quadro 2 — Decisões de incorporação e referência adotadas no projeto

Elemento	Decisão	Justificativa
Itens do pedido	Incorporação	São lidos e gravados junto com o pedido e não existem fora dele
Nome e preço do item	Incorporação como registro histórico	Preserva o valor praticado na data, mesmo que o catálogo mude
Cliente do pedido	Referência por cliente_id	Entidade independente, atualizada por outro fluxo da aplicação
Produto do item	Referência por produto_id	Catálogo compartilhado por muitos pedidos e alterado à parte
Endereço do cliente	Incorporação	Pertence ao cliente e é sempre lido junto com o cadastro
Avaliações do produto	Referência (extensão)	Crescem sem limite e são consultadas de forma independente

Fonte: elaborado pelos autores com base em Marcusso (2026) e em Sadalage e Fowler (2012).

Os compromissos de cada opção são simétricos. A incorporação garante leitura em uma única requisição e atomicidade natural, já que tudo o que muda junto reside no mesmo documento; em troca, duplica informação e faz o documento crescer, sendo o documento ilimitado o principal antipadrão do modelo. A referência mantém o documento enxuto e concentra a atualização em um único ponto, ao custo de requisições adicionais e de uma junção executada na aplicação. Existe ainda o caso híbrido, adotado no item do pedido, em que se guarda a referência ao produto e também uma cópia do nome e do preço no instante da compra: não se trata de redundância acidental, mas de decisão deliberada de historicização, pois alterar o preço do catálogo não deve reescrever o passado.

Uma pergunta prática resume o critério: esse dado muda por conta própria depois de gravado? O preço do catálogo muda, e por isso o produto é referenciado — com cópia do valor no momento da venda. O item do pedido não muda, e por isso é incorporado. Aplicado ao projeto, o pedido incorpora seus itens com o preço do dia, enquanto o cliente permanece referenciado, porque atualiza e-mail e endereço ao longo do tempo sem que nenhum pedido antigo deva ser reescrito por causa disso.

5 QUESTÃO 4 — OS CAMPOS _ID, _REV E TIPO

Explique _id, _rev e tipo.

O campo `_id` é o identificador único do documento dentro do database, imutável após a criação e utilizado diretamente no caminho da requisição HTTP, o que torna a recuperação por chave uma operação trivial. Como o CouchDB não possui coleções, a convenção adotada no projeto é prefixar o identificador com a família do documento, na forma `produto:teclado`, `cliente:uuid` e `pedido:uuid`. O prefixo cumpre três funções: torna a chave legível durante a inspeção no Fauxton, evita colisão entre famílias distintas e permite percorrer documentos de um mesmo tipo por faixa de chave. Adotar identificador significativo para o catálogo e identificador aleatório para clientes e pedidos equilibra legibilidade e geração sem coordenação central.

O campo `_rev` identifica a revisão do documento e é atribuído pelo servidor a cada gravação, no formato composto por um número sequencial e um resumo criptográfico. É o mecanismo de concorrência otimista do banco: toda atualização precisa informar a revisão que o cliente leu e, caso ela não seja mais a corrente, a gravação é recusada com o código 409 e nada é alterado (ANDERSON; LEHNARDT; SLATER, 2010). Convém destacar o que a revisão não é: ela não constitui um histórico de versões da aplicação, pois revisões antigas são descartadas na compactação do banco, e não deve ser fabricada, memorizada por longos períodos ou ignorada pelo código. O fluxo correto consiste em ler o documento, alterá-lo e gravá-lo com a revisão recém-lida.

O campo `tipo`, por sua vez, não possui significado especial para o banco: é um campo comum, definido por convenção do projeto, que atua como discriminador e cumpre o papel funcional da tabela no modelo relacional. Como produtos, clientes e pedidos convivem no mesmo database, praticamente todo seletor Mango começa por esse campo, e todos os índices do projeto o incluem como primeiro elemento. Uma consulta por produtos ativos, por exemplo, filtra simultaneamente por `tipo` e por disponibilidade, e só é eficiente se existir índice composto sobre esses campos. Os três elementos, portanto, dividem responsabilidades bem definidas: `_id` endereça, `_rev` protege contra sobrescrita e `tipo` classifica.

6 QUESTÃO 5 — CONSULTAS MANGO E ÍNDICES

Explique Mango e índices.

Mango é a linguagem declarativa de consulta do CouchDB, escrita em JSON e exposta pelo endpoint `_find`. A consulta é um objeto que contém o seletor, com as condições de filtragem, e opcionalmente a projeção de campos, a ordenação e os limites de paginação. O modelo mental aproxima-se da cláusula de filtro do SQL, com uma diferença decisiva: não há junção, de modo que cada consulta atinge um único database e devolve documentos completos ou projeções deles. Trata-se da alternativa mais acessível às visões escritas em MapReduce, e foi a adotada no projeto por permitir que a mesma estrutura JSON seja escrita no código Python e executada visualmente no Fauxton.

Os índices são criados explicitamente pelo endpoint `_index`, informando os campos na ordem em que serão utilizados. Sem índice adequado a consulta ainda é respondida, porém mediante varredura de todos os documentos — comportamento que passa despercebido com os poucos registros do laboratório e degrada rapidamente em produção, razão pela qual o próprio servidor sinaliza quando nenhum índice corresponde ao seletor. A ordem dos campos é significativa, pois o índice é composto e atende consultas que utilizem um prefixo dele: um índice sobre tipo e categoria serve ao filtro por ambos e ao filtro apenas por tipo, mas não a um filtro apenas por categoria. Índices também consomem espaço e encarecem a escrita, de modo que a regra do projeto é criar um índice por consulta prevista, e não um índice por campo existente.

No projeto, as quatro consultas principais — produtos ativos, cliente por endereço de e-mail, pedidos por cliente e produtos por categoria — originam quatro índices, todos iniciados pelo campo tipo e completados pelo atributo filtrado. Essa correspondência de um para um entre consulta e índice é a tradução prática do desenho orientado à consulta: primeiro se lista o que a aplicação vai perguntar e só então se cria a estrutura capaz de responder. A verificação que encerra a etapa consiste em submeter a consulta ao endpoint de explicação e confirmar que o índice utilizado foi realmente o esperado, evitando a falsa sensação de desempenho que um catálogo pequeno produz.

7 QUESTÃO 6 — POR QUE `_BULK_DOCS` NÃO É UMA TRANSAÇÃO ACID

Por que `_bulk_docs` não é ACID?

O endpoint `_bulk_docs` recebe uma lista de documentos e os grava em uma única requisição HTTP. O ganho é de eficiência de rede, pois reduz o número de idas e voltas entre a aplicação e o banco, e não de atomicidade. Cada documento do lote é avaliado individualmente, e a resposta é um vetor com o resultado de cada um: gravação bem-sucedida com nova revisão, ou recusa por conflito. O lote pode, portanto, ser parcialmente aplicado — alguns documentos gravados e outros recusados —, sem desfazimento automático e sem isolamento, de modo que leitores concorrentes podem observar o estado intermediário.

A razão dessa limitação é arquitetural, e não uma lacuna de implementação. A unidade de atomicidade do CouchDB é o documento, decisão coerente com um banco projetado para replicação entre instâncias que podem operar separadamente e reconciliar depois. Garantir atomicidade sobre vários documentos exigiria coordenação distribuída, com bloqueios e protocolo de confirmação em duas fases, o que comprometeria justamente a disponibilidade e a capacidade de operação desconectada que o modelo busca preservar (KLEPPMANN, 2017). Por isso o material da disciplina insiste na distinção terminológica: chamar o lote de transação sugere uma garantia que o protocolo não oferece, e essa confusão costuma aparecer como defeito somente sob concorrência real.

No checkout do projeto o efeito é concreto. O lote transporta o novo pedido e os documentos de produto com o estoque decrementado. Se um desses produtos tiver sido vendido a outro cliente entre a leitura e a gravação, apenas aquele documento retorna com conflito, enquanto os demais já foram efetivados — resultado possível é um pedido registrado com baixa parcial de estoque. Daí decorrem três práticas obrigatórias: reduzir o número de documentos envolvidos na operação, inspecionar a resposta item a item em vez de assumir sucesso pelo código HTTP da requisição, e prever repetição e compensação para os casos de falha parcial, assunto tratado na questão seguinte.

8 QUESTÃO 7 — TRATAMENTO DO CONFLITO HTTP 409

Como tratar 409?

O código 409 é devolvido pelo CouchDB quando a revisão informada na gravação não é mais a corrente, ou seja, quando outro processo alterou o documento

entre a leitura e a escrita. Não se trata de falha de infraestrutura, e sim do mecanismo de concorrência otimista operando conforme projetado: o banco recusou uma sobrescrita cega e preservou a alteração anterior. O tratamento inadequado é também o mais frequente — capturar a exceção de forma genérica e exibir mensagem de erro interno ao usuário ou, pior, reler o documento e regravá-lo com a nova revisão sem reavaliar a regra de negócio, o que descarta silenciosamente a alteração feita por outro.

O tratamento correto organiza-se em três camadas. A primeira é a releitura com reavaliação: buscar novamente o documento e aplicar a operação sobre o estado atual, e não sobre o estado lido anteriormente, antes de gravar. A segunda é a repetição limitada, com número máximo de tentativas e espera crescente entre elas, pois a repetição imediata em cenário de disputa apenas amplifica as colisões. A terceira é a decisão de negócio quando a reavaliação não se sustenta: se o estoque deixou de existir, o correto é interromper o fluxo e informar o cliente, e não insistir indefinidamente. Operações passíveis de repetição precisam ser idempotentes, com chave de requisição associada, para que uma nova tentativa não gere um segundo pedido.

Reduzir a frequência do conflito também faz parte do tratamento, e é uma decisão de modelagem: separar o contador de estoque do documento de catálogo, ou registrar movimentos de estoque como documentos independentes consolidados posteriormente, diminui a disputa pelo mesmo documento. No projeto, o teste exigido consiste justamente em provocar o conflito, submetendo duas atualizações concorrentes ao mesmo produto por linha de comando ou pelo Fauxton, e verificar que a aplicação responde com mensagem compreensível, sem perder dados e sem duplicar o pedido — evidência que deve constar da entrega.

9 QUESTÃO 8 — COMPENSAÇÃO NO FLUXO DE CHECKOUT

Proponha compensação de checkout.

Como não existe transação entre documentos, o checkout precisa ser desenhado como uma sequência de passos com efeito reversível. A referência conceitual é a decomposição de uma transação longa em subtransações locais, cada qual associada a uma ação compensatória acionada quando um passo posterior falha,

formulação proposta por Garcia-Molina e Salem (1987) e hoje corrente em sistemas distribuídos. O objetivo não é simular atomicidade, e sim garantir que o sistema termine sempre em um estado íntegro e explicável, mesmo quando a operação não se completa.

A proposta para o projeto tem quatro passos. Primeiro, cria-se o documento de pedido com situação inicial de reserva e uma chave de idempotência derivada da sessão e do conteúdo do carrinho, de modo que o pedido funcione como âncora do fluxo e que uma requisição repetida seja reconhecida em vez de duplicada. Segundo, decrementa-se o estoque de cada produto, um documento por vez, registrando no próprio pedido quais itens já tiveram baixa efetivada. Terceiro, concluídas todas as baixas, altera-se a situação do pedido para criado, o que o torna visível no histórico do cliente. Quarto, se qualquer decremento falhar por conflito ou por estoque insuficiente, executa-se a compensação: devolvem-se as quantidades já baixadas, na ordem inversa, e registra-se o pedido como cancelado com o motivo correspondente. Escrever no pedido aquilo que será feito antes de fazê-lo é o que permite retomar ou compensar o fluxo mesmo após uma queda da aplicação no meio do processo.

Três cuidados completam o desenho. A compensação também pode encontrar conflito e, portanto, precisa de releitura e repetição, com as mesmas regras da questão anterior. Ela deve ser idempotente, para que uma execução repetida não devolva estoque duas vezes — exigência atendida pelo registro por item mantido no pedido. E o pedido nunca deve ser removido: um pedido cancelado com motivo registrado é auditável, ao passo que um documento apagado não deixa rastro. Como extensão mais robusta, a reserva de estoque em documento próprio, com prazo de expiração, reduz a janela de conflito e converte a compensação em simples expiração da reserva.

10 QUESTÃO 9 — REPLICAÇÃO NO COUCHDB

Explique replicação.

A replicação no CouchDB é a cópia incremental das alterações de um database de origem para um database de destino, acionada por requisição ao endpoint de replicação ou registrada como documento em um database de controle. O processo percorre o feed de mudanças da origem, identifica as revisões que o destino ainda não possui e transfere apenas essas revisões, o que torna a operação retomável:

interrompida por queda de rede e reiniciada, ela prossegue do ponto em que parou. Pode ser executada uma única vez ou de forma contínua, mantendo o destino sincronizado enquanto houver conexão. A replicação é unidirecional, de modo que a sincronização entre duas instâncias se obtém configurando dois processos em sentidos opostos.

Os usos previstos no projeto são cópia de segurança e recuperação, distribuição entre ambientes e operação desconectada, cenário em que uma instância local funciona sem rede e reconcilia posteriormente (ANDERSON; LEHNARDT; SLATER, 2010). O preço dessa liberdade é o conflito: alterado o mesmo documento nos dois lados, ambas as revisões chegam ao destino e são armazenadas, e o servidor elege deterministicamente uma delas como vencedora — critério arbitrário do ponto de vista do negócio, porém idêntico em todas as réplicas, o que assegura a convergência. A revisão perdedora não é descartada, permanecendo registrada como conflito, e cabe à aplicação detectá-la e resolvê-la segundo a regra de negócio adequada (KLEPPMANN, 2017).

No laboratório, replicar o database do projeto para uma segunda instância, alterar o mesmo produto nos dois lados e observar o conflito resultante no Fauxton é o exercício que torna visível o compromisso entre disponibilidade e consistência. Cabe ainda distinguir replicação de backup: por propagar também exclusões e alterações indevidas, a replicação não protege contra erro humano, o que mantém necessário o instantâneo do volume de dados, com teste periódico de restauração e metas explícitas de ponto e de tempo de recuperação definidas na etapa de operação.

11 QUESTÃO 10 — CONTROLES DE SEGURANÇA PARA HOSPEDAGEM EM NUVEM

Liste controles de segurança cloud.

A hospedagem em nuvem altera o perímetro do sistema: o banco deixa de responder apenas na máquina do laboratório e passa a ser alcançável pela internet, o que torna obrigatórios controles que localmente eram dispensáveis. O ponto de partida é eliminar a configuração padrão de instalação, pois a instância acessível publicamente com credenciais administrativas conhecidas está entre as causas mais recorrentes de incidentes com bancos NoSQL expostos e corresponde às categorias

de configuração insegura e de falha de controle de acesso listadas pelo OWASP (2021). O Quadro 3 organiza os controles mínimos por camada.

Quadro 3 — Controles mínimos de segurança para a implantação do projeto em nuvem

Camada	Controle	Aplicação no projeto
Identidade	Usuário próprio da aplicação, sem privilégio administrativo	Conta com acesso restrito ao database do projeto
Credenciais	Segredos em variáveis de ambiente, fora do repositório	URL de conexão definida no provedor; arquivo de exemplo sem valores reais
Transporte	TLS obrigatório e redirecionamento do tráfego não cifrado	Aplicação ao banco e navegador à aplicação
Rede	Banco não exposto publicamente	Rede privada ou lista de origens autorizadas
Autenticação	Senha armazenada apenas como resumo criptográfico	Geração e verificação do hash pela biblioteca da aplicação
Sessão e entrada	Cookie assinado e validação de entrada e de saída	Chave de sessão secreta e validação dos formulários
Dados pessoais	Minimização e uso de dados fictícios em aula	Somente nome, e-mail e hash de senha
Recuperação	Backup com teste efetivo de restauração	Instantâneo do volume e replicação para outra instância
Observabilidade	Registro e alerta sobre 401, 409 e erros de servidor	Logs da aplicação e métricas do banco

Fonte: elaborado pelos autores com base em Marcusso (2026), na documentação oficial do Apache CouchDB e no OWASP Top 10 (OWASP, 2021).

Alguns desses controles merecem destaque por serem justamente os esquecidos na passagem do laboratório para a nuvem. A URL de conexão do CouchDB carrega usuário e senha embutidos e costuma ser o valor que vaza em um commit descuidado ou em uma mensagem de erro exibida ao usuário, o que justifica mantê-la em variável de ambiente e nunca devolver a exceção original na resposta HTTP. O Fauxton, imprescindível durante o aprendizado, é uma interface administrativa e não deve permanecer acessível publicamente. E a conta utilizada pela aplicação não necessita de poderes administrativos, uma vez que criar bancos, índices e usuários é tarefa da etapa de implantação, e não da execução cotidiana.

Completa o conjunto a disciplina operacional: rotacionar credenciais quando algum integrante deixa o grupo, manter atualizada a imagem do banco, revisar o que os registros de log preservam — nunca senha, token ou documento pessoal — e testar efetivamente a restauração do backup, pois um backup jamais restaurado é apenas uma suposição. Em ambiente de aula, o controle mais simples e mais eficaz continua

sendo o uso exclusivo de dados fictícios: sem dado pessoal real em base, o impacto de uma exposição acidental aproxima-se de zero, o que também dialoga com os princípios de finalidade e de minimização estabelecidos pela legislação de proteção de dados (BRASIL, 2018).

12 CONSIDERAÇÕES FINAIS

O percurso pelas dez questões evidencia que substituir um banco relacional por um banco de documentos não é uma mudança de sintaxe, e sim de repartição de responsabilidades. Aquilo que o PostgreSQL assegura dentro do servidor — integridade referencial, atomicidade sobre várias linhas e verificação de esquema na escrita — passa a ser decidido e implementado pela aplicação quando os dados são documentos: o agregado define a fronteira da atomicidade, a escolha entre incorporar e referenciar define o custo das consultas e a fidelidade histórica, o campo de revisão expõe o conflito no próprio protocolo, e a consistência do checkout converte-se em código de repetição e compensação.

Disso decorre o critério que atravessa todo o projeto: modelar a partir das consultas e do padrão de acesso, e assumir de forma explícita os compromissos implicados na escolha. O CouchDB oferece flexibilidade de formato, leitura por agregado, interface REST e replicação embutida; exige, em contrapartida, índices criados deliberadamente, tratamento de conflito e estratégia própria para operações que envolvem mais de um documento. Reconhecer esse balanço e registrá-lo na documentação da entrega, ao lado dos testes, das evidências e dos controles de segurança, é o que distingue uma escolha técnica justificada de uma preferência tecnológica.

REFERÊNCIAS

ANDERSON, J. Chris; LEHNARDT, Jan; SLATER, Noah. CouchDB: the definitive guide. Sebastopol: O'Reilly Media, 2010.

APACHE SOFTWARE FOUNDATION. Apache CouchDB documentation. Disponível em: <https://docs.couchdb.org/>. Acesso em: 9 set. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

GARCIA-MOLINA, Hector; SALEM, Kenneth. Sagas. In: ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA, 1987, São Francisco. Proceedings [...]. New York: ACM, 1987. p. 249-259.

KLEPPMANN, Martin. Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017.

MARCUSSO, Nivaldo T. Projeto prático NoSQL: e-commerce com Apache CouchDB. Material didático da disciplina Tratamento e Armazenamento de Informação. Campinas: FACAMP, 2026.

OWASP — OPEN WORLDWIDE APPLICATION SECURITY PROJECT. OWASP Top 10: 2021. Disponível em: <https://owasp.org/Top10/>. Acesso em: 9 set. 2026.

SADALAGE, Pramod J.; FOWLER, Martin. NoSQL distilled: a brief guide to the emerging world of polyglot persistence. Upper Saddle River: Addison-Wesley, 2012.

APÊNDICE A — PROMPT UTILIZADO

Reproduz-se a seguir, na íntegra, o prompt utilizado na elaboração deste trabalho com auxílio de inteligência artificial, conforme orientação do professor da disciplina.

PROMPT — QUESTÕES DISSERTATIVAS DO PROJETO PRÁTICO NOSQL COM COUCHDB (ENTREGA ABNT)

CONTEXTO

Você é um assistente acadêmico especializado em bancos de dados e em desenvolvimento de aplicações web. Estou cursando a disciplina Tratamento e Armazenamento da Informação na FACAMP (2026), com o Prof. Me. Nivaldo T. Marcusso. A aula em questão apresenta um projeto prático de e-commerce construído com Python/Flask e Apache CouchDB, percorrendo o ciclo completo: problema e requisitos, modelagem documental (agregados, embed x reference, snapshots), CouchDB e estrutura física (database, `_id`, `_rev`, tipo, índices Mango, Fauxton, Docker), aplicação Python via REST/JSON, consistência e conflitos (`_bulk_docs`, HTTP 409, retry, compensação, idempotência), testes, segurança, backup, replicação, monitoramento e deploy em nuvem. Preciso entregar as 10 questões dissertativas do slide 54 da apresentação, respondidas e formatadas no padrão ABNT.

TAREFA

Responda, de forma dissertativa e fundamentada, às 10 questões abaixo e monte o documento final formatado em ABNT.

1. Justifique CouchDB para o projeto.
2. Compare documento e tabela.
3. Analise embed x reference.
4. Explique `_id`, `_rev` e tipo.
5. Explique Mango e índices.
6. Por que `_bulk_docs` não é ACID?
7. Como tratar 409?
8. Proponha compensação de checkout.
9. Explique replicação.
10. Liste controles de segurança cloud.

COMO RESPONDER

- Extensão: 2 a 3 parágrafos por questão (aproximadamente 200 a 350 palavras cada).
- Estrutura de cada resposta: (a) definição/conceito central; (b) desenvolvimento técnico com trade-offs; (c) exemplo prático concreto, sempre que possível referido ao próprio projeto de e-commerce da disciplina.
- Linguagem: impessoal, acadêmica, em português do Brasil, sem bullet points dentro das respostas (texto corrido dissertativo). Tabelas são permitidas apenas nas questões comparativas e enumerativas (2, 3 e 10), sempre acompanhadas de texto explicativo.
- Consistência com a aula: usar os conceitos apresentados nos slides — modelagem por consulta (query-driven design), agregados, embed x reference, snapshot histórico, prefixo no `_id`, discriminador tipo, MVCC e concorrência otimista por `_rev`, seletores e índices Mango, `_bulk_docs` sem atomicidade multi-documento, HTTP 409, retry, compensação, idempotência, reserva de estoque, replicação one-shot e contínua, Fauxton, segredos em variáveis de ambiente e plano de testes.
- Identificadores técnicos (`_id`, `_rev`, `_bulk_docs`, `_find`, `_index`) devem ser grafados exatamente como aparecem na API; termos em inglês (embed, reference, retry, snapshot) podem ser mantidos em itálico na primeira ocorrência.
- Nada de invenção: não citar fontes, autores, páginas ou números sem certeza de que existem, nem atribuir ao CouchDB comportamento que a documentação não descreve. Se um dado for aproximado, indicar isso no texto.

REFERÊNCIAS

Os slides da aula disponibilizados e a bibliografia abaixo, citada ao longo do texto no formato autor-data ABNT (AUTOR, ano):

- ANDERSON, J. Chris; LEHNARDT, Jan; SLATER, Noah. CouchDB: The Definitive Guide.
- APACHE SOFTWARE FOUNDATION. Apache CouchDB Documentation.
- SADALAGE, Pramod J.; FOWLER, Martin. NoSQL Distilled.
- KLEPPMANN, Martin. Designing Data-Intensive Applications.
- ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados.
- GARCIA-MOLINA, Hector; SALEM, Kenneth. Sagas.
- OWASP. OWASP Top 10.
- BRASIL. Lei nº 13.709/2018 (LGPD).

Usar no mínimo 4 fontes distintas ao longo do trabalho e listar todas — e apenas as efetivamente citadas — na seção de Referências.

FORMATAÇÃO ABNT DO DOCUMENTO

- Capa: FACAMP — Faculdades de Campinas; curso Engenharia da Computação; disciplina "Tratamento e Armazenamento de Informação"; título "Atividade 3.2 — Questões Dissertativas: Projeto Prático NoSQL com Apache CouchDB"; grupo: Felipe Furlan RA 202210339, Henrique Biruel RA 202210347, Henrique Palorca RA 202210348; Campinas; Setembro de 2026.
- Fonte Arial 12, espaçamento entre linhas 1,5, texto justificado, recuo de primeira linha de 1,25 cm.
- Margens: 3 cm superior e esquerda; 2 cm inferior e direita.
- Numeração de páginas no canto superior direito.
- Seções numeradas: cada questão vira uma seção (1 INTRODUÇÃO, 2 QUESTÃO 1 — título curto do tema, ..., 12 CONSIDERAÇÕES FINAIS, REFERÊNCIAS).
- Enunciado de cada questão reproduzido em itálico logo abaixo do título da seção, antes da resposta.
- Citações diretas curtas entre aspas com (AUTOR, ano, p. X); citações longas (mais de 3 linhas) com recuo de 4 cm, fonte 10, espaçamento simples, sem aspas.
- Quadros numerados sequencialmente, com título acima e indicação de fonte abaixo.
- Referências ao final, em ordem alfabética, alinhadas à esquerda, espaçamento simples e separadas por uma linha em branco.
- Incluir uma Introdução curta (1 parágrafo) e Considerações finais (1 a 2 parágrafos amarrando a ideia de que o modelo documental desloca para a aplicação responsabilidades que o banco relacional garante internamente).
- Ao final do documento, acrescentar um Apêndice A — Prompt utilizado, reproduzindo integralmente este prompt (exigência do professor).

ENTREGA

Gerar o resultado em .docx e em PDF, no padrão ABNT, com o nome de arquivo: TAI - Atividade 3.2 - Questões Dissertativas Projeto NoSQL CouchDB - Grupo 3. Ao terminar, enviar os arquivos e um resumo de uma linha do que foi produzido.